NTT与分治NTT

NTT

模板：P3803 【模板】多项式乘法（FFT） - 洛谷

2024CCPC重庆G：魔弹 - Problem - QOJ.ac

2024ICPC昆明I：Items - Problem - QOJ.ac

2025ICPC武汉L：ICPC - Problem - QOJ.ac

2025ICPC南京M：Many Convex Polygons - Problem - QOJ.ac

分治 + NTT

模板：P4721 【模板】分治 FFT - 洛谷

2024ICPC上海F：Fast Bogosort - Problem - QOJ.ac

2025CCPC济南M：MEX Problem - Problem - QOJ.ac

2026ICPC深圳邀请赛B：All-Star Showdown - Problem - QOJ.ac

[模板]多项式乘法（FFT）

题意：给定一个 n 次多项式 $F(x)$ 和一个 m 次多项式 $G(x)$，求 $F(x)$ 和 $G(x)$ 的乘积

数据范围：$1 \le n, m \le 10^6$

时间复杂度：$O(n \log n)$

```
#define mod 998244353
typedef long long ll;
struct mint{int _;mint(){_=0;}mint(ll a){_=a%mod;if(_<0)_+=mod;}void read()
{cin>>_;}};
mint operator+(mint a,mint b){a._+=b._;if(a._>=mod)a._-=mod;return a;}
mint operator-(mint a,mint b){a._-=b._;if(a._<0)a._+=mod;return a;}
mint operator*(mint a,mint b){a._=(int)((ll)a._*b._%mod);return a;}
mint operator+=(mint& a,mint b){a._+=b._;if(a._>=mod)a._-=mod;return a;}
mint operator-=(mint& a,mint b){a._-=b._;if(a._<0)a._+=mod;return a;}
mint operator*=(mint& a,mint b){a._=(int)((ll)a._*b._%mod);return a;}
mint pow(mint a,int b){mint c=1;while(b>0){if(b&1)c*=a;a*=a;b>>=1;}return c;}
mint inv(mint a){return pow(a,mod-2);}
const int N = 4000010;  // 数组大小要开 (n + m) * 2

namespace NTT {
```

```
    mint g = 3;
    mint gi = inv(g);
    int rev[N];

    void ntt(mint A[], int tot, int type) {
        for(int i = 0; i < tot; i++)
            if(i < rev[i]) swap(A[i], A[rev[i]]);
        for(int mid = 1; mid < tot; mid <<= 1) {
            mint gn = pow(type > 0 ? g : gi, (mod - 1) / (mid << 1));
            int R = mid << 1;
            for(int i = 0; i < tot; i+=R) {
                mint g0 = 1;
                for(int j = i; j < mid + i; j++, g0*= gn) {
                    mint x = A[j], y = g0 * A[j + mid];
                    A[j] = x + y;
                    A[j + mid] = x - y;
                }
            }
        }
        if(type == -1) {
            mint invtot = inv(tot);
            for(int i = 0; i < tot; i++)
                A[i] *= invtot;
        }
    }

    void polymul_ntt(mint a[], mint b[], int n, int m) {
        int tot = 1, l = 0;
        while(tot <= n + m) {
            tot <<= 1, l ++;
        }
        for(int i = 0; i <= tot; i++)
            rev[i] = 0;
        for(int i = 0; i <= tot; i++)
            rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (l - 1));
        ntt(a, tot, 1);
        ntt(b, tot, 1);
        for(int i = 0; i <= tot; i++)
            a[i] *= b[i];
        ntt(a, tot, -1);
        ntt(b, tot, -1);
    }
}

int n, m;
mint a[N], b[N];

void solve() {
```

```
    cin >> n >> m;
    for(int i = 0; i <= n; i++)
        cin >> a[i]._;
    for(int i = 0; i <= m; i++)
        cin >> b[i]._;
    NTT::polymul_ntt(a, b, n, m);
    for(int i = 0; i <= n + m; i++)
        cout << a[i]._ << " \n"[i == n + m];
}
```

[模板]分治FFT

题意：给定序列 $g_{1,\ldots,n-1}$，求序列 $f_{0,\ldots,n-1}$。其中 $f_i = \sum_{j=1}^{i} f_{i-j} g_j$，边界为 $f_0 = 1$，答案对 998244353 取模

数据范围：$2 \le n \le 10^5$，$0 \le g_i < 998244353$

题解：分治 NTT，可以用 CDQ 分治，先求出 $f_{[l,mid)}$，可以发现这部分对区间 $f_{[mid,r)}$ 的贡献是 $f_{[l,mid)} * g_{[0,r-l)}$，卷出来加到对应位置即可

时间复杂度：$O(n \log_2^2 n)$

```
#define mod 998244353
typedef long long ll;
struct mint{int _;mint(){_=0;}mint(ll a){_=a%mod;if(_<0)_+=mod;}void read()
{cin>>_;}};
mint operator+(mint a,mint b){a._+=b._;if(a._>=mod)a._-=mod;return a;}
mint operator-(mint a,mint b){a._-=b._;if(a._<0)a._+=mod;return a;}
mint operator*(mint a,mint b){a._=(int)((ll)a._*b._%mod);return a;}
mint operator+=(mint& a,mint b){a._+=b._;if(a._>=mod)a._-=mod;return a;}
mint operator-=(mint& a,mint b){a._-=b._;if(a._<0)a._+=mod;return a;}
mint operator*=(mint& a,mint b){a._=(int)((ll)a._*b._%mod);return a;}
mint pow(mint a,int b){mint c=1;while(b>0){if(b&1)c*=a;a*=a;b>>=1;}return c;}
mint inv(mint a){return pow(a,mod-2);}
const int N = 6000010;  // 数组大小要开 (n + m) * 2

namespace NTT {
    mint g = 3;
    mint gi = inv(g);
    int rev[N];

    void ntt(mint A[], int tot, int type) {
        for(int i = 0; i < tot; i++)
            if(i < rev[i]) swap(A[i], A[rev[i]]);
        for(int mid = 1; mid < tot; mid <<= 1) {
            mint gn = pow(type > 0 ? g : gi, (mod - 1) / (mid << 1));
```

```
            int R = mid << 1;
            for(int i = 0; i < tot; i+=R) {
                mint g0 = 1;
                for(int j = i; j < mid + i; j++, g0*= gn) {
                    mint x = A[j], y = g0 * A[j + mid];
                    A[j] = x + y;
                    A[j + mid] = x - y;
                }
            }
        }
        if(type == -1) {
            mint invtot = inv(tot);
            for(int i = 0; i < tot; i++)
                A[i] *= invtot;
        }
    }

    void polymul_ntt(mint a[], mint b[], int n, int m) {
        int tot = 1, l = 0;
        while(tot <= n + m) {
            tot <<= 1, l ++;
        }
        for(int i = 0; i <= tot; i++)
            rev[i] = 0;
        for(int i = 0; i <= tot; i++)
            rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (l - 1));
        ntt(a, tot, 1);
        ntt(b, tot, 1);
        for(int i = 0; i <= tot; i++)
            a[i] *= b[i];
        ntt(a, tot, -1);
        ntt(b, tot, -1);
    }
}

int n;
int f[N], g[N];
mint a[N], b[N];

void cdq_solve(int l, int r) {
    if(l == r) return;
    int mid = l + r >> 1;
    cdq_solve(l, mid);

    int len = mid - l + 1;
    for(int i = 0; i <= len * 6; i++)
        a[i] = b[i] = 0;
    for(int i = 1; i <= len; i++)
```

```
        a[i] = f[l + i - 1];
    for(int i = 0; i <= len * 2; i++)
        b[i] = g[i];
    NTT::polymul_ntt(a, b, len, len * 2);
    for(int i = mid + 1; i <= r; i++)
        f[i] = (f[i] + a[i - l + 1]._) % mod;
    cdq_solve(mid + 1, r);
}

void solve() {
    cin >> n;
    for(int i = 1; i < n; i++)
        cin >> g[i];
    f[0] = 1;
    cdq_solve(0, n - 1);
    for(int i = 0; i < n; i++)
        cout << f[i] << " \n"[i + 1 == n];
}
```

All-Star Showdown

题意：有 n 名选手，每位选手有三维坐标 (x_i, y_i, z_i) 和一个权值 w_i 。将所有人分成红、蓝两队，要求每名选手恰好属于一支队伍，同一支队伍内任意两名选手的欧几里得距离 $\geq d$ ，一个队伍的价值为队伍内所有成员的权值的乘积。对 $k = 1, 2, \cdots, n-1$ ，求所有合法分组里，红队恰好 k 人时，红队价值的总和，答案对 998244353 取模

数据范围：$2 \leq n \leq 10^5$ ，$1 \leq d \leq 10^8$ ，$1 \leq x_i, y_i, z_i, w_i \leq 10^8$

题解：整道题分为二分图判定+生成函数计数两部分。首先，在欧几里得距离小于 d 的选手之间连边，题目要求红队和蓝队内部都没有边，等价于该图必须是二分图。考虑如何高效建图，将整个几何空间划分为无数个边长为 d 的立方体，即将点 (x, y, z) 映射到 $(\lfloor \frac{x}{d} \rfloor, \lfloor \frac{y}{d} \rfloor, \lfloor \frac{z}{d} \rfloor)$ 的立方体中，在满足二分图的前提下，每个立方体内最多只能存在 2 个点，且两个距离小于 d 的点必然在同一个立方体或相邻的 26 个立方体中。将点压缩映射后，暴力枚举相邻立方体中的点，最多存在 $2 \times 27 \times n$ 条边，随后进行二分图染色即可。如果是二分图，每个连通分量有两种染色方案，我们用生成函数来合并所有连通分量的选择，计算出红队人数为 k 时的总权值和，这一步可以用分治NTT进行加速

时间复杂度：$O(54n + n \log^2 n)$

```
const int N = 1000010;
const int mod = 998244353;

int n, m;
int a[N], b[N], c[N], w[N];
vector<int> e[N];

```

```cpp
bool build_graph() {
    map<array<int,3>,int> mp;
    int d = m, cnt = 0;
    for(int i = 1; i <= n; i++) {
        array<int,3> t = {a[i] / d, b[i] / d, c[i] / d};
        if(mp.find(t) == mp.end())
            mp[t] = cnt ++;
    }
    vector<vector<int>> ve(cnt, vector<int>());
    for(int i = 1; i <= n; i++) {
        array<int,3> t = {a[i] / d, b[i] / d, c[i] / d};
        int id = mp[t];
        ve[id].push_back(i);
    }
    for(auto ve1: ve)
        if(ve1.size() > 2)
            return false;

    auto check = [&](int u, int v) -> bool {
        if(u == v) return false;
        int dx = a[u] - a[v], dy = b[u] - b[v], dz = c[u] - c[v];
        return dx * dx + dy * dy + dz * dz < m * m;
    };
    for(int id = 0; id < cnt; id++) {
        vector<int> ve1 = ve[id];
        for(auto u: ve1) {
            int x = a[u] / d, y = b[u] / d, z = c[u] / d;
            for(int i = x - 1; i <= x + 1; i++)
                for(int j = y - 1; j <= y + 1; j++)
                    for(int k = z - 1; k <= z + 1; k++)
                        if(mp.find({i, j, k}) != mp.end()) {
                            int id2 = mp[{i, j, k}];
                            vector<int> ve2 = ve[id2];
                            for(auto v: ve2)
                                if(check(u, v))
                                    e[u].push_back(v);
                        }
        }
    }
    return true;
}

int tot;
int f[N], g[N], f_val[N], g_val[N];
bool bipartite_graph() {
    for(int i = 1; i <= n; i++)
        if(e[i].size() > 2)
            return false;
```

```

    tot = 0;
    vector<int> col(n + 1, -1);
    vector<int> id(n + 1);

    auto dfs = [&](auto&& self, int u, int c) -> bool {
        col[u] = c, id[u] = tot;
        for(auto v: e[u]) {
            if(col[v] == -1) {
                if(!self(self, v, !c))
                    return false;
            } else if(col[v] == c) {
                return false;
            }
        }
        return true;
    };

    for(int i = 1; i <= n; i++)
        if(col[i] == -1) {
            tot ++;
            if(!dfs(dfs, i, 0))
                return false;
        }

    for(int i = 1; i <= tot; i++) {
        f[i] = g[i] = 0;
        f_val[i] = g_val[i] = 1;
    }
    for(int i = 1; i <= n; i++)
        if(col[i] == 0) {
            f[id[i]] ++;
            f_val[id[i]] = f_val[id[i]] * w[i] % mod;
        } else {
            g[id[i]] ++;
            g_val[id[i]] = g_val[id[i]] * w[i] % mod;
        }
    return true;
}

#define mod 998244353
typedef long long ll;
struct mint{int _;mint(){_=0;}mint(ll a){_=a%mod;if(_<0)_+=mod;}void read()
{cin>>_;}};
mint operator+(mint a,mint b){a._+=b._;if(a._>=mod)a._-=mod;return a;}
mint operator-(mint a,mint b){a._-=b._;if(a._<0)a._+=mod;return a;}
mint operator*(mint a,mint b){a._=(int)((ll)a._*b._%mod);return a;}
mint operator+=(mint& a,mint b){a._+=b._;if(a._>=mod)a._-=mod;return a;}
```

```
mint operator-=(mint& a,mint b){a._-=b._;if(a._<0)a._+=mod;return a;}
mint operator*=(mint& a,mint b){a._=(int)((ll)a._*b._%mod);return a;}
mint pow(mint a,int b){mint c=1;while(b>0){if(b&1)c*=a;a*=a;b>>=1;}return c;}
mint inv(mint a){return pow(a,mod-2);}

namespace NTT {
    mint g = 3;
    mint gi = inv(g);
    int rev[N];

    void ntt(mint A[], int tot, int type) {
        for(int i = 0; i < tot; i++)
            if(i < rev[i]) swap(A[i], A[rev[i]]);
        for(int mid = 1; mid < tot; mid <<= 1) {
            mint gn = pow(type > 0 ? g : gi, (mod - 1) / (mid << 1));
            int R = mid << 1;
            for(int i = 0; i < tot; i+=R) {
                mint g0 = 1;
                for(int j = i; j < mid + i; j++, g0*= gn) {
                    mint x = A[j], y = g0 * A[j + mid];
                    A[j] = x + y;
                    A[j + mid] = x - y;
                }
            }
        }
        if(type == -1) {
            mint invtot = inv(tot);
            for(int i = 0; i < tot; i++)
                A[i] *= invtot;
        }
    }

    void polymul_ntt(mint a[], mint b[], int n, int m) {
        int tot = 1, l = 0;
        while(tot <= n + m) {
            tot <<= 1, l ++;
        }
        for(int i = 0; i <= tot; i++)
            rev[i] = 0;
        for(int i = 0; i <= tot; i++)
            rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (l - 1));
        ntt(a, tot, 1);
        ntt(b, tot, 1);
        for(int i = 0; i <= tot; i++)
            a[i] *= b[i];
        ntt(a, tot, -1);
        ntt(b, tot, -1);
    }
```

```
}

mint yzq[N], yx[N];
vector<int> cdq_solve(int l, int r) {
    if(l == r) {
        int sz = max(f[l], g[l]);
        vector<int> res(sz + 1, 0);
        res[f[l]] = (res[f[l]] + f_val[l]) % mod;
        res[g[l]] = (res[g[l]] + g_val[l]) % mod;
        return res;
    }
    int mid = l + r >> 1;
    auto L = cdq_solve(l, mid), R = cdq_solve(mid + 1, r);
    int sz1 = L.size() - 1, sz2 = R.size() - 1;
    int sz = sz1 + sz2;

    for(int i = 0; i <= sz * 4; i++)
        yzq[i] = yx[i] = 0;
    for(int i = 0; i <= sz1; i++)
        yzq[i] = L[i];
    for(int i = 0; i <= sz2; i++)
        yx[i] = R[i];
    NTT::polymul_ntt(yzq, yx, sz1, sz2);
    vector<int> res(sz + 1);
    for(int i = 0; i <= sz; i++)
        res[i] = yzq[i]._;
    return res;
}

void solve() {
    cin >> n >> m;
    for(int i = 1; i <= n; i++) {
        e[i].clear();
        cin >> a[i] >> b[i] >> c[i] >> w[i];
    }

    if(!build_graph()) {
        for(int i = 1; i < n; i++)
            cout << "0\n";
        return;
    }
    if(!bipartite_graph()) {
        for(int i = 1; i < n; i++)
            cout << "0\n";
        return;
    }

    vector<int> res = cdq_solve(1, tot);
```

```
    res.resize(n);
    for(int i = 1; i < n; i++)
        cout << res[i] << '\n';
}
```

Items

题意：给定 n 种物品，每种物品有无限个，重量在 0 ~ n 中。判断能否选出恰好 n 个物品，使得总重量恰好为 m

数据范围：$n \le 10^5$ ，$m \le n^2$

题解：设 b_i 表示是否存在重量为 i 的物品，定义生成函数：$f(x) = \sum\limits_{i=0}^{n} b_i x^i$ ，那么答案为 $[x^m]f(x)^n > 0$ 。直接计算 $f(x)^n$ 显然是不行的，不妨令 $B = \lfloor \frac{m}{n} \rfloor$ ，$w_i \leftarrow w_i - B$ ，$m \leftarrow m - nB$ 。显然，对 w 和 m 变换后的问题与原问题是等价的。此时，$w_i \in [-B, n-B]$ ，$m \in [0, n]$

可以通过倍增的方式计算 $f(x)^n$ ，且每次倍增后只需要保留 x^{-n} ~ x^n 中的项即可。上述过程使用 FFT 或 NTT 实现均可，需要每次倍增后都将系数二值化，否则会导致浮点溢出

时间复杂度：$O(n \log^2 n)$

```
#define mod 998244353
typedef long long ll;
struct mint{int _;mint(){_=0;}mint(ll a){_=a%mod;if(_<0)_+=mod;}void read()
{cin>>_;}};
mint operator+(mint a,mint b){a._+=b._;if(a._>=mod)a._-=mod;return a;}
mint operator-(mint a,mint b){a._-=b._;if(a._<0)a._+=mod;return a;}
mint operator*(mint a,mint b){a._=(int)((ll)a._*b._%mod);return a;}
mint operator+=(mint& a,mint b){a._+=b._;if(a._>=mod)a._-=mod;return a;}
mint operator-=(mint& a,mint b){a._-=b._;if(a._<0)a._+=mod;return a;}
mint operator*=(mint& a,mint b){a._=(int)((ll)a._*b._%mod);return a;}
mint pow(mint a,int b){mint c=1;while(b>0){if(b&1)c*=a;a*=a;b>>=1;}return c;}
mint inv(mint a){return pow(a,mod-2);}
const int N = 1600010;

namespace NTT {
    mint g = 3;
    mint gi = inv(g);
    int rev[N];

    void ntt(mint A[], int tot, int type) {
        for(int i = 0; i < tot; i++)
            if(i < rev[i]) swap(A[i], A[rev[i]]);
```

```
        for(int mid = 1; mid < tot; mid <<= 1) {
            mint gn = pow(type > 0 ? g : gi, (mod - 1) / (mid << 1));
            int R = mid << 1;
            for(int i = 0; i < tot; i+=R) {
                mint g0 = 1;
                for(int j = i; j < mid + i; j++, g0*= gn) {
                    mint x = A[j], y = g0 * A[j + mid];
                    A[j] = x + y;
                    A[j + mid] = x - y;
                }
            }
        }
        if(type == -1) {
            mint invtot = inv(tot);
            for(int i = 0; i < tot; i++)
                A[i] *= invtot;
        }
    }

    void polymul_ntt(mint a[], mint b[], int n, int m) {
        int tot = 1, l = 0;
        while(tot <= n + m) {
            tot <<= 1, l ++;
        }
        for(int i = 0; i <= tot; i++)
            rev[i] = 0;
        for(int i = 0; i <= tot; i++)
            rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (l - 1));
        ntt(a, tot, 1);
        ntt(b, tot, 1);
        for(int i = 0; i <= tot; i++)
            a[i] *= b[i];
        ntt(a, tot, -1);
        ntt(b, tot, -1);
    }
}

mint a[N], b[N], res[N];

void solve() {
    int n, m;
    cin >> n >> m;
    vector<int> w(n);
    for(int i = 0; i < n; i++)
        cin >> w[i];
    int B = m / n;
    for(int i = 0; i < n; i++)
        w[i] -= B;
```

```
    m -= n * B;

    // res: [-2n, 2n] -> [0, 4n]
    // a: [-2n, 2n] -> [0, 4n]
    // res * a: [-4n, 4n] -> [0, 8n]
    // a * a: [-4n, 4n] -> [0, 8n]
    for(int i = 0; i <= n * 16; i++)
        a[i] = res[i] = 0;
    res[n * 2] = 1;
    for(int i = 0; i < n; i++)
        a[w[i] + n * 2] = 1;

    int x = n;
    while(x) {
        if(x & 1) {
            for(int i = n * 4 + 1; i <= n * 16; i++)
                res[i] = 0;
            for(int i = 0; i <= n * 4; i++)
                b[i] = a[i];
            for(int i = n * 4 + 1; i <= n * 16; i++)
                b[i] = 0;
            NTT::polymul_ntt(res, b, n * 4, n * 4);
            for(int i = 0; i <= n * 4; i++)
                res[i]._ = res[i + n * 2]._ ? 1 : 0;
        }
        for(int i = 0; i <= n * 4; i++)
            b[i] = a[i];
        for(int i = n * 4 + 1; i <= n * 16; i++)
            a[i] = b[i] = 0;
        NTT::polymul_ntt(a, b, n * 4, n * 4);
        for(int i = 0; i <= n * 4; i++)
            a[i]._ = a[i + n * 2]._ ? 1 : 0;
        x >>= 1;
    }
    if(res[m + n * 2]._) cout << "Yes\n";
    else cout << "No\n";
}
```

Many Convex Polygons

题意：按照逆时针顺序给出凸 n 边形的顶点，选 k 个不同的顶点顺次连成凸 k 边形，对所有选法计算求凸 k 边形面积之和的两倍，对每个 $k = 3, 4, \ldots, n$ 计算答案对 998244353 取模后的结果

数据范围：$3 \le n \le 2 \times 10^5$ ，$-10^9 \le x_i, y_i \le 10^9$

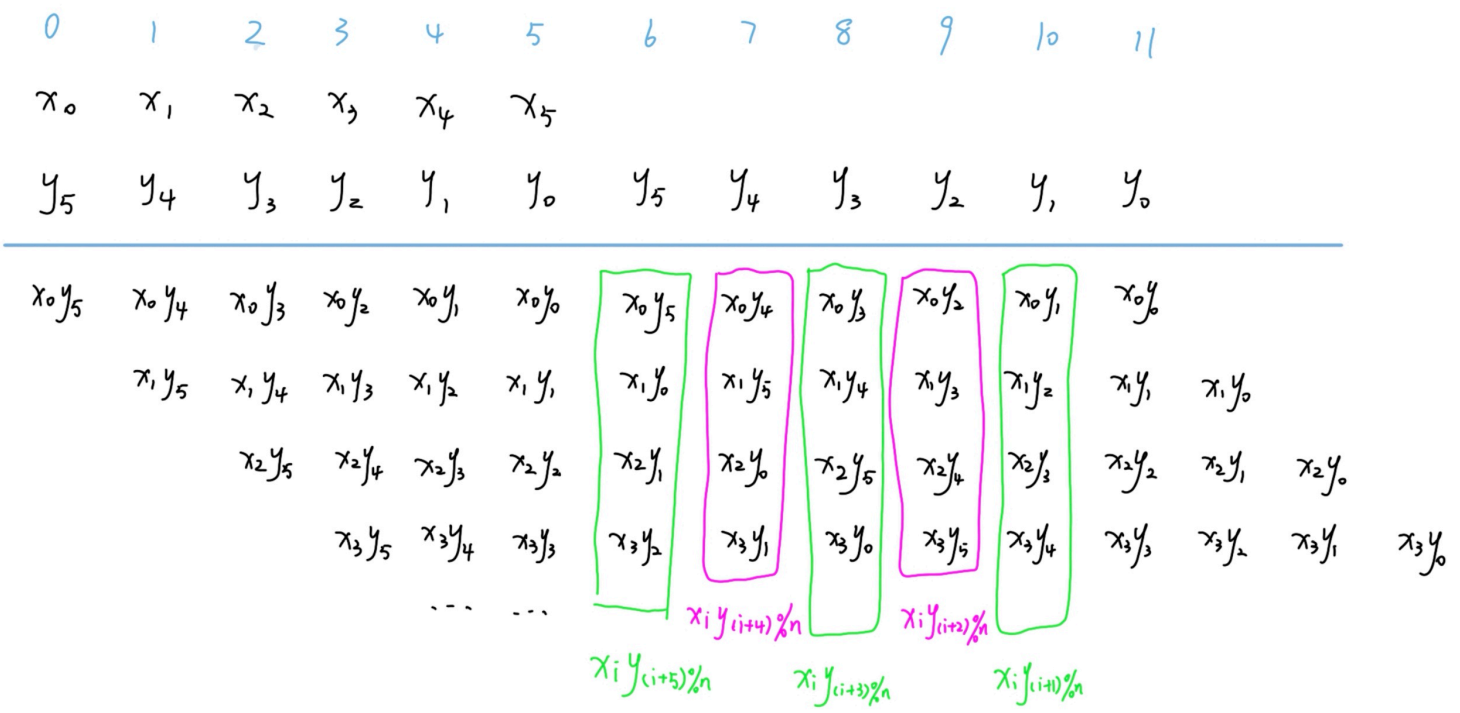
题解：记选出的 k 个点为 $P(i_0), P(i_1), \cdots, P(i_{k-1})$，那么凸 k 边形面积的两倍是

$$\sum_{j=0}^{k-1} P(i_j) \times P(i_{(j+1) \bmod k}) = \sum_{j=0}^{k-1} (x_{i_j} y_{i_{(j+1) \bmod k}} - y_{i_j} x_{i_{(j+1) \bmod k}})$$

考虑线段 $P(i)P((i+d) \bmod n)$ 作为凸 k 边形的边，线段右侧的 $d-1$ 个点都不能作为凸 k 边形的顶点，还要从左侧的 $n-1-d$ 个点中选出 $k-2$ 个点作为凸 k 边形的顶点，那么这条线段对 ans_k 的贡献是 $\binom{n-1-d}{k-2}(x_i y_{(i+d) \bmod n} - y_i x_{(i+d) \bmod n})$

对 i 求和，记 $f_d = \sum_{i=0}^{n-1} (x_i y_{(i+d) \bmod n} - y_i x_{(i+d) \bmod n})$，使用 NTT 优化循环差卷积即可求出所有 $f_1, f_2, \ldots, f_{n-2}$

于是有 $ans_k = \sum_{d=1}^{n-1} \binom{n-1-d}{k-2} f_d = \frac{1}{(k-2)!} \sum_{d=1}^{n-k+1} \frac{(n-1-d)! f_d}{(n-k+1-d)!}$，使用 NTT 优化差卷积即可求出所有 $ans_3, ans_4, \ldots, ans_n$



令 $g_d = (n-1-d)!\, f_d$

$$\frac{g_d}{(n-k+1-d)!} \to ans_k$$

g_1	g_2	g_3	g_4
$\frac{1}{0!}$	$\frac{1}{1!}$	$\frac{1}{2!}$	$\frac{1}{3!}$
$\frac{g_1}{0!} \to ans_6$	$\frac{g_1}{1!} \to ans_5$	$\frac{g_1}{2!} \to ans_4$	$\frac{g_1}{3!} \to ans_3$
	$\frac{g_2}{0!} \to ans_5$	$\frac{g_2}{1!} \to ans_4$	$\frac{g_2}{2!} \to ans_3$
		$\frac{g_3}{0!} \to ans_4$	$\frac{g_3}{1!} \to ans_3$
			$\frac{g_4}{0!} \to ans_3$

$$\frac{g_1}{(6-k)!} \to ans_k$$

$$\frac{g_2}{(5-k)!} \to ans_k$$

$$\frac{g_3}{(4-k)!} \to ans_k$$

$$\frac{g_4}{(3-k)!} \to ans_k$$

时间复杂度：$O(n \log n)$

```cpp
#define mod 998244353
typedef long long ll;
const int N = 1200010;
struct mint{int _;mint(){_=0;}mint(ll a){_=a%mod;if(_<0)_+=mod;}void read()
{cin>>_;}};
mint operator+(mint a,mint b){a._+=b._;if(a._>=mod)a._-=mod;return a;}
mint operator-(mint a,mint b){a._-=b._;if(a._<0)a._+=mod;return a;}
mint operator*(mint a,mint b){a._=(int)((ll)a._*b._%mod);return a;}
mint operator+=(mint& a,mint b){a._+=b._;if(a._>=mod)a._-=mod;return a;}
mint operator-=(mint& a,mint b){a._-=b._;if(a._<0)a._+=mod;return a;}
mint operator*=(mint& a,mint b){a._=(int)((ll)a._*b._%mod);return a;}
mint pow(mint a,int b){mint c=1;while(b>0){if(b&1)c*=a;a*=a;b>>=1;}return c;}
mint inv(mint a){return pow(a,mod-2);}
mint fact[N],infact[N];
mint C(int a,int b){if(b<0||b>a)return 0;return fact[a]*infact[b]*infact[a-b];}
mint invC(int a,int b){if(b<0||b>a)return 0;return infact[a]*fact[b]*fact[a-b];}
void initC(int n){fact[0]=1;for(int i=1;i<=n;i++)fact[i]=fact[i-1]*i;
infact[n]=inv(fact[n]);for(int i=n;i>=1;i--)infact[i-1]=infact[i]*i;}

namespace NTT {
    mint g = 3;
    mint gi = inv(g);
    int rev[N];

    void ntt(mint A[], int tot, int type) {
```

```
        for(int i = 0; i < tot; i++)
            if(i < rev[i]) swap(A[i], A[rev[i]]);
        for(int mid = 1; mid < tot; mid <<= 1) {
            mint gn = pow(type > 0 ? g : gi, (mod - 1) / (mid << 1));
            int R = mid << 1;
            for(int i = 0; i < tot; i+=R) {
                mint g0 = 1;
                for(int j = i; j < mid + i; j++, g0*= gn) {
                    mint x = A[j], y = g0 * A[j + mid];
                    A[j] = x + y;
                    A[j + mid] = x - y;
                }
            }
        }
        if(type == -1) {
            mint invtot = inv(tot);
            for(int i = 0; i < tot; i++)
                A[i] *= invtot;
        }
    }

    void polymul_ntt(mint a[], mint b[], int n, int m) {
        int tot = 1, l = 0;
        while(tot <= n + m) {
            tot <<= 1, l ++;
        }
        for(int i = 0; i <= tot; i++)
            rev[i] = 0;
        for(int i = 0; i <= tot; i++)
            rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (l - 1));
        ntt(a, tot, 1);
        ntt(b, tot, 1);
        for(int i = 0; i <= tot; i++)
            a[i] *= b[i];
        ntt(a, tot, -1);
        ntt(b, tot, -1);
    }
}

mint a1[N], b1[N], a2[N], b2[N];
mint f[N], g[N];

void solve() {
    int n;
    cin >> n;
    vector<int> x(n), y(n);
    for(int i = 0; i < n; i++)
        cin >> x[i] >> y[i];
```

```

    // xi * y(i + d)
    for(int i = 0; i < n; i++) {
        a1[i] = x[i];
        b1[i] = b1[i + n] = y[n - i - 1];
    }
    NTT::polymul_ntt(a1, b1, n, n * 2);

    // yi * x(i + d)
    for(int i = 0; i < n; i++) {
        a2[i] = y[i];
        b2[i] = b2[i + n] = x[n - i - 1];
    }
    NTT::polymul_ntt(a2, b2, n, n * 2);

    for(int i = 1; i + 1 < n; i++)
        f[i] = a1[n * 2 - i - 1] - a2[n * 2 - i - 1];

    initC(n);
    for(int i = 1; i + 1 < n; i++)
        f[i] *= fact[n - 1 - i];
    for(int i = 0; i + 1 < n; i++)
        g[i] = infact[i];
    NTT::polymul_ntt(f, g, n, n);

    for(int i = 3; i <= n; i++) {
        mint ans = f[n - i + 1];
        ans *= infact[i - 2];
        cout << ans._ << '\n';
    }
}
```

Fast Bogosort

题意：给定一个特定的随机排序算法：每次将序列切割成尽量多块，使得每一块内包含所有各自该有的数字，每一块独立区间 $shuffle$ 后递归执行该算法。给定初始排列，求排列完成时区间 $shuffle$ 次数的期望

数据范围：$1 \le n \le 10^5$

题解：枚举分拆的第一分段点，把一个切 k 刀的任务分解成切 1 刀 + 切 $k-1$ 刀。令 $g(n)$ 表示长度为 n 不可分割的排列数量，有转移方程 $g(n) = n! - \sum\limits_{m=0}^{n-1} g(m)(n-m)!$ 。含义是全排列数减去可分割的排列数量，后者进一步枚举第一分段点 m ，前 m 个数不可分割，后 $n-m$ 个数任选，有 $g(m)(n-m)!$ 种方法。令 $f(n)$ 表示长度为 n 的随机排列的答案，

$f(n)=1+\frac{1}{n!}\sum_{m=0}^{n}g(m)(n-m)!(f(m)+f(n-m)-\mathbb{I}(n-m\neq 1))$。求和符号里的含义表示，枚举第一分段点 m 有 $g(m)(n-m)!$ 种方法，后面乘上的是递归的代价，分为两部分，前 m 个数就是 $f(m)$，后 $n-m$ 个数是 $f(n-m)-\mathbb{I}(n-m\neq 1)$。等号右边也有 $f(n)$ 项，整理为严格递推式

$$f(n)=\frac{1}{n!-g(n)}(n!+\sum_{m=0}^{n-1}g(m)(n-m)!(f(m)+f(n-m)-\mathbb{I}(n-m\neq 1)))$$

现在需要算两个式子：

$$g(n)=n!-\sum_{m=0}^{n-1}g(m)(n-m)!$$

$$f(n)=\frac{1}{n!-g(n)}(n!+\sum_{m=0}^{n-1}g(m)(n-m)!(f(m)+f(n-m)-\mathbb{I}(n-m\neq 1)))$$

对式子进行简单变形，整理成CDQ分治多项式乘法的形式：

$$g(n)=n!-\sum_{m=0}^{n-1}g(m)(n-m)!$$

$$f(n)=\frac{1}{n!-g(n)}(n!-\sum_{m=0}^{n-2}g(m)(n-m)!+\sum_{m=0}^{n-1}f(m)g(m)(n-m)!+\sum_{m=0}^{n-1}f(m)m!g(n-m))$$

时间复杂度：$O(n\log^2 n)$

```
#define mod 998244353
typedef long long ll;
const int N = 800010;
struct mint{int _;mint(){_=0;}mint(ll a){_=a%mod;if(_<0)_+=mod;}void read()
{cin>>_;}};
mint operator+(mint a,mint b){a._+=b._;if(a._>=mod)a._-=mod;return a;}
mint operator-(mint a,mint b){a._-=b._;if(a._<0)a._+=mod;return a;}
mint operator*(mint a,mint b){a._=(int)((ll)a._*b._%mod);return a;}
mint operator+=(mint& a,mint b){a._+=b._;if(a._>=mod)a._-=mod;return a;}
mint operator-=(mint& a,mint b){a._-=b._;if(a._<0)a._+=mod;return a;}
mint operator*=(mint& a,mint b){a._=(int)((ll)a._*b._%mod);return a;}
mint pow(mint a,int b){mint c=1;while(b>0){if(b&1)c*=a;a*=a;b>>=1;}return c;}
mint inv(mint a){return pow(a,mod-2);}
mint fact[N],infact[N];
mint C(int a,int b){if(b<0||b>a)return 0;return fact[a]*infact[b]*infact[a-b];}
mint invC(int a,int b){if(b<0||b>a)return 0;return infact[a]*fact[b]*fact[a-b];}
void initC(int n){fact[0]=1;for(int i=1;i<=n;i++)fact[i]=fact[i-1]*i;
infact[n]=inv(fact[n]);for(int i=n;i>=1;i--)infact[i-1]=infact[i]*i;}

namespace NTT {
    mint g = 3;
    mint gi = inv(g);
    int rev[N];
```

```cpp

    void ntt(mint A[], int tot, int type) {
        for(int i = 0; i < tot; i++)
            if(i < rev[i]) swap(A[i], A[rev[i]]);
        for(int mid = 1; mid < tot; mid <<= 1) {
            mint gn = pow(type > 0 ? g : gi, (mod - 1) / (mid << 1));
            int R = mid << 1;
            for(int i = 0; i < tot; i+=R) {
                mint g0 = 1;
                for(int j = i; j < mid + i; j++, g0*= gn) {
                    mint x = A[j], y = g0 * A[j + mid];
                    A[j] = x + y;
                    A[j + mid] = x - y;
                }
            }
        }
        if(type == -1) {
            mint invtot = inv(tot);
            for(int i = 0; i < tot; i++)
                A[i] *= invtot;
        }
    }

    void polymul_ntt(mint a[], mint b[], int n, int m) {
        int tot = 1, l = 0;
        while(tot <= n + m) {
            tot <<= 1, l ++;
        }
        for(int i = 0; i <= tot; i++)
            rev[i] = 0;
        for(int i = 0; i <= tot; i++)
            rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (l - 1));
        ntt(a, tot, 1);
        ntt(b, tot, 1);
        for(int i = 0; i <= tot; i++)
            a[i] *= b[i];
        ntt(a, tot, -1);
        ntt(b, tot, -1);
    }
}

int n;
int f[N], g[N];
mint a[N], b[N];

void cdq_solveg(int l, int r) {
    if(l == r) return;
    int mid = l + r >> 1, len = mid - l + 1;
```

```
    cdq_solveg(l, mid);

    for(int i = 0; i <= len * 6; i++)
        a[i] = b[i] = 0;
    for(int i = 1; i <= len; i++)
        a[i] = g[l + i - 1];
    for(int i = 0; i < len * 2; i++)
        b[i] = fact[i + 1];
    NTT::polymul_ntt(a, b, len, len * 2);
    for(int i = mid + 1; i <= r; i++)
        g[i] = (g[i] - a[i - mid + len - 1]._ + mod) % mod;

    cdq_solveg(mid + 1, r);
}

void cdq_solvef(int l, int r) {
    if(l == r) {
        int t = (fact[l]._ - g[l] + mod) % mod;
        f[l] = f[l] * inv(t)._ % mod;
        return;
    }
    int mid = l + r >> 1, len = mid - l + 1;
    cdq_solvef(l, mid);

    for(int i = 0; i <= len * 6; i++)
        a[i] = b[i] = 0;
    for(int i = 1; i <= len; i++)
        a[i] = g[l + i - 1];
    for(int i = 1; i < len * 2; i++)
        b[i] = fact[i + 1];
    NTT::polymul_ntt(a, b, len, len * 2);
    for(int i = mid + 1; i <= r; i++)
        f[i] = (f[i] - a[i - mid + len - 1]._ + mod) % mod;

    for(int i = 0; i <= len * 6; i++)
        a[i] = b[i] = 0;
    for(int i = 1; i <= len; i++)
        a[i] = f[l + i - 1] * g[l + i - 1] % mod;
    for(int i = 0; i < len * 2; i++)
        b[i] = fact[i + 1];
    NTT::polymul_ntt(a, b, len, len * 2);
    for(int i = mid + 1; i <= r; i++)
        f[i] = (f[i] + a[i - mid + len - 1]._) % mod;

    for(int i = 0; i <= len * 6; i++)
        a[i] = b[i] = 0;
    for(int i = 1; i <= len; i++)
        a[i] = f[l + i - 1] * fact[l + i - 1]._ % mod;
```

```
    for(int i = 0; i < len * 2; i++)
        b[i] = g[i + 1];
    NTT::polymul_ntt(a, b, len, len * 2);
    for(int i = mid + 1; i <= r; i++)
        f[i] = (f[i] + a[i - mid + len - 1]._) % mod;

    cdq_solvef(mid + 1, r);
}

void solve() {
    cin >> n;

    initC(n);
    for(int i = 1; i <= n; i++)
        f[i] = g[i] = fact[i]._;
    cdq_solveg(1, n);
    cdq_solvef(1, n);

    int res = 0, la = 0, mx = 0;
    for(int i = 1, x; i <= n; i++) {
        cin >> x;
        mx = max(mx, x);
        if(mx == i) {
            res = (res + f[i - la]) % mod;
            la = i;
        }
    }
    cout << res << '\n';
}
```